```
class Camera
  render: ->
    if /'Lacia'/.test @name
      "Special deal"
```

If the name of this object looks like 'Lacia', then it's on special.

It's not going to be that easy. Agtron wants to change the special deals dynamically. You don't know in advance what's going to be discounted, so you can't declare it in the class. You'll need to extend the class dynamically. To learn how to extend a class dynamically, you need to learn how classes really work. It's finally time to see the JavaScript that the CoffeeScript compiler creates from a class declaration.

The compiled JavaScript for a CoffeeScript class is shown in the following listing. As always, CoffeeScript is on the left and compiled JavaScript is on the right.

Listing 5.6. CoffeeScript class and compilation to JavaScript

CoffeeScript

```
class Simple
  constructor: ->
    @name = "simple object"

simple = new Simple
```

JavaScript

```
var Simple = (function() {
  function Simple() {
    this.name = 'simple';
  }
  return Simple;
})();

simple = new Simple();
```

Inside a CoffeeScript class you see functions. That explains the variable scope. CoffeeScript classes have a variable scope like functions because they *are* functions. One second, though; where did the constructor go?

Constructor functions

In JavaScript a constructor is a function that creates a new object and links it to a prototype object. In addition to the prototype, all objects have a reference to their constructor function. In JavaScript when you use the `new` keyword in front of a function, an object is created and the function is invoked with the `this` keyword inside the function (known as `@` in